



[http://java.sun.com/developer/technicalArticles/J2SE/jvm\\_ti/](http://java.sun.com/developer/technicalArticles/J2SE/jvm_ti/)

Jan 28, 2008

## Article

# The JVM Tool Interface (JVM TI): How VM Agents Work

By Kelly O'Hair and Janice J. Heiss, December 2006

The JVM tool interface (JVM TI) is a standard native API that allows for native libraries to capture events and control a Java Virtual Machine (JVM) for the Java platform.\* These native libraries are sometimes called agent libraries and are often used as a basis for the Java technology-level tool APIs, such as the Java Debugger Interface (JDI) that comes with the Java Development Kit (JDK). Profiler tool vendors will often need to create an agent library that uses JVM TI. This article explores some basics of writing a JVM TI agent library by walking through the `heapTracker` demo agent available in the JDK downloads.

In the releases prior to JDK 5.0, an agent was loaded into a virtual machine (VM) at initialization with the option `-XrunNAME`, where `NAME` is the name of the native shared library or DLL, such as `libNAME.so` or `NAME.dll`. For example, when using `HPROF` and inputting `java -Xrunhprof`, the library `libhprof.so` or `hprof.dll` would be found in the JDK. The VM would cause that library to be dynamically loaded, and the VM would make a call into that library to get it started. The option and timing of when these libraries were loaded were nonstandard. The `-Xrun` option would load the libraries after the VM had initialized itself, sometimes resulting in early VM events not being available to the agent. The library itself would use the Java Native Interface (JNI) and either JVM Debug Interface (JVMDI) for debugging or the experimental JVM Profiling Interface (JVMPI) for profiling. Both of these are being removed from future JDK releases. Here is [an article on the transition from JVMPI to JVM TI](#).

Beginning with JDK 5.0, the new standard option is `-agentlib`, for example, `java -agentlib:hprof`, although JDK 5.0 still accepts the `-Xrun` option. The new agent-loading options are documented and official. The library is loaded before the VM has initialized, allowing the agent library to capture early VM events that it could not access before. The library itself then uses JVM TI and JNI for debugging, profiling, or doing anything an agent does. A set of sample JVM TI agents is available in the `demo` directory of the [JDK 5.0](#) or the [JDK 6](#) download. Source and binaries are included for those interested in creating their own custom agent library.

## Buyer Beware: Don't Accept Agents From Strangers

Because the agent library will be operating in the same process and address space as the VM itself, anything inside the agent code will run in the VM process too. A bad agent can crash the entire VM process with a trivial null pointer dereference. Agents can also be very difficult to get right. Agent libraries must be re-entrant and MT-safe, and they must follow all the JVM TI and JNI rules. For instance, if your agent leaks memory by calling `malloc()` and not doing the `free()`, then the VM will appear to have a leak. Allocating too much memory will cause the VM process to fail with an out of memory error. You must pay close attention to detail when you add an agent library.

## Native Library Loading

The VM process must be able to locate the native library by way of the platform-specific search rules, so you must either copy the library into your JDK with the other shared libraries or make it accessible through a platform-specific mechanism so that a process can locate it. For example, you can use `LD_LIBRARY_PATH` on the Solaris Operating Environment or Linux operating system, or you can use `PATH` on Microsoft Windows. In addition, the agent library must be able to locate all the external symbols it needs from any platform-specific shared libraries. On the Solaris and Linux operating systems, you can use the `ldd` utility to verify that a native library knows how to find all the necessary externals. Once the VM process has successfully loaded an agent library, it looks for a symbol in it to call and establish the agent-to-VM connection. The native library should have exposed an exported symbol with the name `Agent_OnLoad`. This will be the first function called in the agent library.

## The Dynamic Tracing (DTrace) Agent

An example VM agent of note is the Dynamic Tracing (DTrace) agent located at the [Solaris 10 OS DTrace VM agents project](#). The `dvm.zip` file includes the built libraries and the source code to the Solaris OS JVM TI agent.

DTrace is a comprehensive dynamic tracing framework for the Solaris OS that provides a powerful infrastructure to permit administrators, developers, and service personnel to concisely answer arbitrary questions about the behavior of

the OS and user programs. The [Solaris Dynamic Tracing Guide](#) describes how to use DTrace to observe, debug, and tune system behavior. The guide also includes a complete reference for bundled DTrace observability tools and the D programming language.

The JDK 6 release includes built-in DTrace probes, whereas older JDKs such as 5.0 or 1.4.2 can be limited with respect to DTrace. The VM agent for use with Solaris 10 OS dynamic tracing ([DVM agent](#)) is useful for older JDK releases in that it indirectly provides DTrace probes inside the agent library itself. The DVM agent is unusual in that it requests VM events but often does nothing itself with the event besides providing a DTrace probe point. Though this creates some unnecessary overhead, the functionality it provides can be extremely valuable.

Again, the [JDK 6 release](#) includes virtually all required built-in DTrace probes, thus eliminating the need for a DVM agent.

## Agent Interfaces

Great care must be taken when writing agents because exposing your agent requires that you have a well-planned testing strategy and are familiar with highly recursive and highly re-entrant coding.

In general, byte-code instrumentation (BCI) is the recommended way to instrument class files, and BCI is easy to do in JVM TI. BCI provides a way to inject code into the class file methods, either before the VM sees the class file (`ClassLoaderHook`) or by redefining the class files on the fly (`RedefineClass`). See [this blog entry](#) for more information about BCI. Note: JVM TI does not provide code to do the BCI. Instead, its goal is to allow you to replace class files that have been BCI'd. See the section "[BCI and BCI Events](#)" later in this article for more information.

Depending on your needs, it may be helpful to get a basic understanding of what agent interfaces can and cannot do. The documentation is worth browsing to become familiar with these interfaces. Look at the documentation for the following JDKs:

- [JDK 5.0 agent API and JNI](#)
- [JDK 6 agent API and JNI](#)

## Agent Initialization

After the VM has located your shared library and successfully loaded it into the VM process, it looks in the library for `Agent_OnLoad`. JVM TI has capabilities that must be requested during the `Agent_OnLoad` execution. This better informs the VM what the agent will need to do and allows for optimal performance based on the capabilities you have requested. To avoid unnecessary overhead in the VM, agents generally should request only the capabilities they need.

So what does agent initialization look like? Following is some code to illustrate. Note: This sample code has been shortened to make it easier to follow, so it is incomplete. The code is mainly from the `heapTracker` demo JVM TI agent that is shipped with JDK 5.0 or more recent versions. To find all the details and comments, get the complete copy of `heapTracker.c` in the `demo/jvmti/heapTracker` directory of any JDK 5.0 or JDK 6 binary download.

```
#include "jvmti.h"
#include "jni.h"

static jrawMonitorID agent_lock;

JNIEXPORT jint JNICALL
Agent_OnLoad(JavaVM *vm, char *options, void *reserved) {
    jvmtiEnv    *jvmti;
    jvmtiError   error;
    jint         res;
    jvmtiCapabilities capabilities;
    jvmtiEventCallbacks callbacks;

    // Create the JVM TI environment (jvmti).
    res = (*vm)->GetEnv(vm, (void **)&jvmti, JVMTI_VERSION_1);
    // If res!=JNI_OK generate an error.

    // Parse the options supplied to this agent on the command line.
    parse_agent_options(options);
    // If options don't parse, do you want this to be an error?

    // Clear the capabilities structure and set the ones you need.
    (void)memset(&capabilities,0, sizeof(capabilities));
```

```

capabilities.can_generate_all_class_hook_events = 1;
capabilities.can_tag_objects = 1;
capabilities.can_generate_object_free_events = 1;
capabilities.can_get_source_file_name = 1;
capabilities.can_get_line_numbers = 1;
capabilities.can_generate_vm_object_alloc_events = 1;

// Request these capabilities for this JVM TI environment.
error = (*jvmti)->AddCapabilities(jvmti, &capabilities);
// If error!=JVMTI_ERROR_NONE, your agent may be in trouble.

// Clear the callbacks structure and set the ones you want.
(void)memset(&callbacks,0, sizeof(callbacks));
callbacks.VMStart = &cbVMStart;
callbacks.VMInit = &cbVMInit;
callbacks.VMDeath = &cbVMDeath;
callbacks.ObjectFree = &cbObjectFree;
callbacks.VMObjectAlloc = &cbVMObjectAlloc;
callbacks.ClassFileLoadHook = &cbClassFileLoadHook;
error = (*jvmti)->SetEventCallbacks(jvmti, &callbacks,
                                   (jint)sizeof(callbacks));
// If error!=JVMTI_ERROR_NONE, the callbacks were not accepted.

// For each of the above callbacks, enable this event.
error = (*jvmti)->SetEventNotificationMode(jvmti, JVMTI_ENABLE,
                                           JVMTI_EVENT_VM_START, (jthread)NULL);
error = (*jvmti)->SetEventNotificationMode(jvmti, JVMTI_ENABLE,
                                           JVMTI_EVENT_VM_INIT, (jthread)NULL);
error = (*jvmti)->SetEventNotificationMode(jvmti, JVMTI_ENABLE,
                                           JVMTI_EVENT_VM_DEATH, (jthread)NULL);
error = (*jvmti)->SetEventNotificationMode(jvmti, JVMTI_ENABLE,
                                           JVMTI_EVENT_OBJECT_FREE, (jthread)NULL);
error = (*jvmti)->SetEventNotificationMode(jvmti, JVMTI_ENABLE,
                                           JVMTI_EVENT_VM_OBJECT_ALLOC, (jthread)NULL);
error = (*jvmti)->SetEventNotificationMode(jvmti, JVMTI_ENABLE,
                                           JVMTI_EVENT_CLASS_FILE_LOAD_HOOK,
                                           (jthread)NULL);
// In all the above calls, check errors.

// Create a raw monitor in the agent for critical sections.
error = (*jvmti)->CreateRawMonitor(jvmti, "agent data",
                                   &(agent_lock));
// If error!=JVMTI_ERROR_NONE, then you haven't got a lock!

return JNI_OK; // Indicates to the VM that the agent loaded OK.
}

```

Note: The authors have ignored the error returns in this code sample, *not* a good practice. Do not copy the preceding code sample without adding checks on the error returns. In JVM TI, because agents run inside the VM itself, the developer should set up extensive error checking. Any error in an agent probably indicates a problem in the implementation of your agent and is something that you should address. Errors in the case of `Agent_OnLoad` should either cause the process to exit or should print an error and disable itself. It is important to decide what your agent should do in this situation.

## Event Callbacks

What do you do once you've set up `Agent_OnLoad` with the proper capabilities and event requests? Once the VM is running, you should start seeing calls to the functions you supplied to `SetEventCallbacks`. In this particular case, a naming convention, `cb` prefix, is used with these functions: `cbVMStart`, `cbVMInit`, `cbObjectFree`, `cbVMObjectAlloc`, `cbClassFileLoadHook`, and `cbVMDeath`. The `cbClassFileLoadHook` is called for each class file image being loaded, and it will likely be called first, at least until the first few basic system classes are loaded before `cbVMStart`.

## JVMTI\_EVENT\_CLASS\_FILE\_LOAD\_HOOK

Let's now look at `cbClassFileLoadHook`, which is called before any class is loaded and, thus, will probably be called first:

```

static void JNICALL
cbClassFileLoadHook(jvmtiEnv *jvmti, JNIEnv* env,
                    jclass class_being_redefined, jobject loader,
                    const char* name, jobject protection_domain,
                    jint class_data_len, const unsigned char* class_data,
                    jint* new_class_data_len,
                    unsigned char** new_class_data)
{
    enterCriticalSection(jvmti); {

        // Safety check, if VM is dead, skip this.
        if ( !gdata->vmDead ) {
            const char * classname;

            // If you have no classname, dig it out of the class file.
            if ( name == NULL ) {
                classname = java_crw_demo_classname(class_data,
                                                    class_data_len, NULL);
            } else {
                classname = strdup(name);
            }

            // Assume you won't change the class file at first.
            *new_class_data_len = 0;
            *new_class_data     = NULL;

            // Be careful that you don't track the tracker class.
            if (strcmp(classname, STRING(HEAP_TRACKER_class))!=0) {
                jint      cnum;
                int       systemClass;
                unsigned char *newImage;
                long       newLength;

                // Processed class counter
                cnum = gdata->ccount++;

                // Tell java_crw_demo if this is an early class.
                systemClass = 0;
                if ( !gdata->vmStarted ) {
                    systemClass = 1;
                }

                // Use java_crw_demo to create a new class file.
                newClassData = NULL;
                newLength = 0;
                java_crw_demo(cnum, classname, class_data,
                            class_data_len, systemClass,
                            STRING(HEAP_TRACKER_class),
                            "L" STRING(HEAP_TRACKER_class) ";",
                            NULL, NULL, NULL, NULL,
                            STRING(HEAP_TRACKER_newobj),
                            "(Ljava/lang/Object;)V",
                            STRING(HEAP_TRACKER_newarr),
                            "(Ljava/lang/Object;)V",
                            &newClassData, &newLength, NULL, NULL);

                // If it did something, make a JVM TI copy.
                if ( newLength > 0 ) {
                    unsigned char *jvmti_space;
                    jvmti_space = (unsigned char *)
                        allocate(jvmti, (jint)newLength);
                    (void)memcpy(jvmti_space,
                                newClassData, newLength);
                    *new_class_data_len = (jint)newLength;
                    *new_class_data     = jvmti_space;
                }

                // Free any malloc space created.
                if ( newClassData != NULL ) {
                    (void)free((void*)newClassData);
                }
            }
        }
    }
}

```

```

    }

    // Free the classname (malloc space too).
    (void)free((void*)classname);
}
} exitCriticalSection(jvmti);
}

```

The function `java_crw_demo` is a pure native and independent non-JNI library function that this article discusses in the section "[BCI and BCI Events](#)." Note that `java_crw_demo` accepts class data bytes and returns new class data bytes in memory. Nothing in the agent can disturb the global static data (`gdata`) while the classload is processing. In effect, the classload has not occurred. Rather, this event represents the time during which the VM has located the class file and read it into memory, but before it has processed the class data bytes. During this event, the bytes that represent the class can be replaced, and the VM will load the replacement class data bytes. The `java_crw_demo` library is limited in terms of what gets changed in the class. It will not add methods, fields, or arguments to methods; nor will it change the basic interface or shape of the object. The intent here is to instrument the existing method byte codes.

Because some classes are loaded before the VM start event, what this callback does is quite important. This agent has requested these `ClassFileLoadHook` events from the beginning, so it needs to be very careful what it does if the VM has not started or been initialized before the callbacks have been made.

Notice that the test on `gdata->vmDead` offers protection if another thread is trying to terminate the VM. There is no need to process class files if VM death is imminent.

The `classname NULL` occurs rarely and only occurs when the `ClassLoader.defineClass()` method is used with a `NULL` name. When that happens, a `java_crw_demo` library function gets the name from the class file.

The new class data bytes may include calls to the `Tracker` class (see the section "[BCI and BCI Events](#)"), so the `strcmp()` on the `HEAP_TRACKER_CLASS` is very important. If you inject calls to the `HEAP_TRACKER_CLASS` inside the `HEAP_TRACKER_CLASS`, you will create an infinite loop.

The `gdata->ccount` allows a unique numeric ID for every class loaded. This is passed into the main `java_crw_demo` function.

Finally, note the use of `gdata->vmStarted`. A better solution might be coming, but for now, the first classes loaded between `Agent_OnLoad` and the `VM_START` event are considered, for lack of a better term, system classes. The `java_crw_demo` treats these classes, of which there are usually 12 or fewer, in a special way when instrumenting them due to their primordial nature and the state of the VM prior to the VM start event. For more on this subject, check the details of `java_crw_demo` in the `demo/jvmti` directory.

The memory allocated by the `java_crw_demo` library is `malloc()` memory; it is not JVM TI-allocated memory. The VM gets the new class data bytes through the arguments `new_class_data_len` and `new_class_data`. The memory returned back to the VM must be allocated by way of `JVM TI Allocate`, which is why the `malloc()` allocated `java_crw_demo` memory is copied. The `java_crw_demo` code is neutral code and does not have any dependence on JVM TI or the VM. It's a C library with standard C library dependencies.

## JVMTI\_EVENT\_VM\_START

After some core system classes are loaded and the VM has started but not completely initialized, the `VM_START` event is posted. You can then call many JNI functions. However, because the VM is not fully initialized, there are limitations in what can be done at this point. At the VM start event, the VM is considered to be out of its primordial phase.

```

static void JNICALL
cbVMStart(jvmtiEnv *jvmti, JNIEnv *env) {
    enterCriticalSection(jvmti); {
        jclass klass;
        jfieldID field;
        jint rc;
        static JNINativeMethod registry[2] = {
            {STRING(HEAP_TRACKER_native_newobj),

```

```

        "(Ljava/lang/Object;Ljava/lang/Object;)V",
        (void*)&HEAP_TRACKER_native_newobj },
        {STRING(HEAP_TRACKER_native_newarr),
        "(Ljava/lang/Object;Ljava/lang/Object;)V",
        (void*)&HEAP_TRACKER_native_newarr }
    };

    // Find the tracker class.
    klass = (*env)->FindClass(env, STRING(HEAP_TRACKER_class));

    // Register the native methods to the ones in this library.
    rc = (*env)->RegisterNatives(env, klass, registry, 2);

    // Get the static field "engaged" in this class.
    field = (*env)->GetStaticFieldID(env, klass,
        STRING(HEAP_TRACKER_engaged), "I");

    // Set the value of this static field to "1."
    (*env)->SetStaticIntField(env, klass, field, 1);

    // Record that the VM has officially started.
    gdata->vmStarted = JNI_TRUE;

    } exitCriticalSection(jvmti);
}

```

For the VM start event, you must first set up the Tracker class used for BCI. First, the JNI function `FindClass` is called to get the `jclass` handle. Note that this could trigger a `ClassFileLoadHook` event. Then the native methods are registered for the Tracker class with JNI `RegisterNatives`, and the `jfieldID` handle to `engaged` is obtained using JNI `GetStaticFieldID`. By setting this static field to 1, you essentially activate the calls inside this Tracker class. The Tracker source looks like this:

```

public class HeapTracker {

    // The static field that controls tracking
    private static int engaged = 0;

    // Calls to this method will result in a call into the agent.
    private static native void _newobj(Object thread, Object o);

    // Calls to this method are injected into the class files.
    public static void newobj(Object o) {
        if ( engaged != 0 ) {
            _newobj(Thread.currentThread(), o);
        }
    }

    // Calls to this method will result in a call into the agent.
    private static native void _newarr(Object thread, Object a);

    // Calls to this method are injected into the class files.
    public static void newarr(Object a) {
        if ( engaged != 0 ) {
            _newarr(Thread.currentThread(), a);
        }
    }
}

```

All the Tracker methods that will be called by the BCI classes modified by `java_crw_demo` are turned off by default until the `engaged` field value changes to 1. When this occurs, it triggers the injected Tracker Method calls to call the native methods that have already registered. Before this article discusses what occurs next in the native methods, note that these native calls cannot be turned on until the VM has started. It's necessary to be able to call JNI functions in the native code, which is why `engaged` was set to 1 here and not earlier. If the above Tracker code were to initiate

anything more sophisticated, such as calling methods in classes, that would require waiting until the VM initialization phase.

### TraceInfo and Tracker Methods

The `heapTracker` agent's main function is to find out what is allocating the most space. At each object allocation, it's important to find out what the stack trace is and then to tag the object with a reference to that trace information. Inside the agent itself, there is a `TraceInfo` struct, and pointers to these structs will serve as the value of the `Object Tags`. A *tag* is any 64-bit value. Along with the `TraceInfo` struct is support code to create a hash table for quick lookups. Because the creation or lookup of the `TraceInfo` will probably consume much application time when this agent is activated, speed and efficiency are important. Here are the basics to find `TraceInfo`:

```
static TraceInfo *
findTraceInfo(jvmtiEnv *jvmti, jthread thread, TraceFlavor flavor) {
    TraceInfo *tinfo;
    jvmtiError error;

    tinfo = NULL;

    // The thread could be NULL in some situations, so be careful.
    if ( thread != NULL ) {
        static Trace  empty;
        Trace        trace;

        // Request a stack trace.
        trace = empty;
        error = (*jvmti)->GetStackTrace(jvmti, thread, 0,
                                      MAX_FRAMES+2,
                                      trace.frames, &(trace.nframes));

        // If you get a PHASE error, the VM isn't ready, or it died.
        if ( error == JVMTI_ERROR_WRONG_PHASE ) {
            if ( flavor == TRACE_USER ) {
                tinfo = emptyTrace(TRACE_BEFORE_VM_INIT);
            } else {
                tinfo = emptyTrace(flavor);
            }
        } else {
            // If error!=JVMTI_ERROR_NONE, you have serious problems.
            check_jvmti_error(jvmti, error, "Cannot get stack trace");
            // Look up this entry.
            tinfo = lookupOrEnter(jvmti, &trace, flavor);
        }
    } else {
        // If thread==NULL, it's assumed this is before VM_START.
        // But technically this should not happen, no tracking yet.
        if ( flavor == TRACE_USER ) {
            tinfo = emptyTrace(TRACE_BEFORE_VM_START);
        } else {
            tinfo = emptyTrace(flavor);
        }
    }
    return tinfo;
}
```

If `thread==NULL`, that usually means that the VM initialization has not yet occurred, so you cannot get a stack trace. Calling `GetStackTrace` could also return an error message that the VM is not in the `live` phase, but a value of `JVMTI_ERROR_NONE` indicates that you did get a stack trace. Performing a `lookupOrEnter` of this stack trace into the hash table will return a reference to a `TraceInfo` structure. This pointer to a `TraceInfo` struct will then be used as the tag on this object. All objects allocated from the same stack trace will have the same tag.

When saving away stack trace information, pay attention to the `TraceInfo` struct and the number of stack traces received. There should be only one stack trace of an allocation byte code (`new` or `new array` byte code), but how many stack traces will that be? It depends on the application, but it's somewhat limited. Also, no cleanup is occurring in

lookupOrEnter, so a very long-running application could experience some problems if the total number of allocation traces is very high. The hash table is also a fixed size, another potential problem. To avoid performance issues, being aware of the critical sections, one of which is in the lookupOrEnter() function, is crucial. Using too many critical sections could dramatically slow the entire application. The thread used here is the current user thread; limit what you are doing in these threads.

Object allocations and object-free events are fairly critical-section free with regard to this particular agent.

### JVMTI\_EVENT\_VM\_INIT

After the VM\_START event and approximately several hundred class load events, the VM will reach the fully initialized event.

```
static void JNICALL
cbVMInit(jvmtiEnv *jvmti, JNIEnv *env, jthread thread) {
    jvmtiError error;

    // Iterate over the entire heap and tag untagged objects.
    error = (*jvmti)->IterateOverHeap(jvmti,
                                       JVMTI_HEAP_OBJECT_UNTAGGED,
                                       &cbObjectTagger, NULL);

    enterCriticalSection(jvmti); {
        gdata->vmInitialized = JNI_TRUE;
    } exitCriticalSection(jvmti);
}
```

Despite full initialization and setting gdata->vmInitialized, many objects were allocated but were not tracked because the Tracker classes are not turned on until the agent gets the VMStart event. Using the JVM TI IterateOverHeap to traverse the heap, objects can now be tagged. Remember: You cannot track objects unless you tag them.

### JVMTI\_EVENT\_OBJECT\_FREE

```
static void JNICALL
cbObjectFree(jvmtiEnv *jvmti, jlong tag)
{
    TraceInfo *tinfo;

    // Don't bother if dead.
    if ( gdata->vmDead ) {
        return;
    }

    // The object tag is actually a pointer to a TraceInfo struct.
    tinfo = (TraceInfo*)(void*)(ptrdiff_t)tag;

    // Decrement the use count.
    tinfo->useCount--;
}
```

### JVMTI\_EVENT\_VM\_OBJECT\_ALLOC

```
static void JNICALL
cbVMOBJECTAlloc(jvmtiEnv *jvmti, JNIEnv *env, jthread thread,
                jobject object, jclass object_klass, jlong size)
{
    TraceInfo *tinfo;
```



```

    // Don't bother if dead.
    if ( gdata->vmDead ) {
        return;
    }

    // Create a stack trace and tag the object.
    tinfo = findTraceInfo(jvmti, thread, TRACE_VM_OBJECT);
    tagObjectWithTraceInfo(jvmti, object, tinfo);
}

```

## JVMTI\_EVENT\_VM\_DEATH

As the name implies, the JVM TI event named VM death is the last VM event. However, due to multiple threads, other event callbacks could still be in progress during this event callback. Depending on thread priorities, it's hard to predict the timing of the code in these final callbacks. Some agents use a lock and a counter to keep track of the active callbacks, then wait in this VM death callback until the count reaches zero. Be careful here, and don't assume that all event callbacks are completed.

```

static void JNICALL
cbVMDeath(jvmtiEnv *jvmti, JNIEnv *env) {
    jvmtiError error;

    // IterateOverHeap can see garbage, so force a GC first.
    error = (*jvmti)->ForceGarbageCollection(jvmti);

    // Notice that you hold no locks on this call, that's important.
    error = (*jvmti)->IterateOverHeap(jvmti,
                                      JVMTI_HEAP_OBJECT_EITHER,
                                      &cbObjectSpaceCounter, NULL);

    enterCriticalSection(jvmti); {
        jclass          klass;
        jfieldID        field;
        jvmtiEventCallbacks callbacks;

        // Find the heap tracker class.
        klass = (*env)->FindClass(env, STRING(HEAP_TRACKER_class));

        // Get the static "engaged" field.
        field = (*env)->GetStaticFieldID(env, klass,
                                          STRING(HEAP_TRACKER_engaged), "I");

        // Set the engaged field to "0," turns off BCI calls in
        // Tracker class.
        (*env)->SetStaticIntField(env, klass, field, 0);

        // Clear the callbacks struct and clear the JVM TI callbacks.
        (void)memset(&callbacks, 0, sizeof(callbacks));
        error = (*jvmti)->SetEventCallbacks(jvmti, &callbacks,
                                           (jint)sizeof(callbacks));

        // Consider the VM dead at this point.
        gdata->vmDead = JNI_TRUE;
        if ( gdata->traceInfoCount > 0 ) {
            TraceInfo **list;
            int          count;
            int          i;

            // Allocate space for a sorted list of TraceInfos.
            stdout_message("Dumping heap trace information\n");
            list = (TraceInfo**)calloc(gdata->traceInfoCount,
                                      sizeof(TraceInfo*));

            count = 0;
            for ( i = 0 ; i < HASH_BUCKET_COUNT ; i++ ) {
                TraceInfo *tinfo;

```

```

        tinfo = gdata->hashBuckets[i];
        while ( tinfo != NULL ) {
            if ( count < gdata->traceInfoCount ) {
                list[count++] = tinfo;
            }
            tinfo = tinfo->next;
        }
    }

    // Sort the list and print out the top ones.
    qsort(list, count, sizeof(TraceInfo*), &compareInfo);
    for ( i = 0 ; i < count ; i++ ) {
        if ( i >= gdata->maxDump ) {
            break;
        }
        printTraceInfo(jvmti, i+1, list[i]);
    }

    // Free the space you allocated.
    (void)free(list);
}
} exitCriticalSection(jvmti);
}

```

To summarize, first use JVM TI to force garbage collection in order to iterate over the heap and get a count, per stack trace, of the objects currently allocated. Next, turn the `Tracker` class off and disconnect all the JVM TI callbacks, keeping in mind that some callbacks may still be active and that you have turned off any future callbacks by removing their addresses from the JVM TI environment. You can accomplish the same thing by disabling the events. Finally, construct a single-dimensioned list of all the `TraceInfo` structures, sort it by allocation amount, and print out up to `gdata->maxDump` of the stack traces that allocated the most memory.

## Object Tagging

As you can see, working with VM agent code can be challenging. Look at the higher-level view to get a better grasp. This example code, called `heapTracker`, exists to track all object allocations in the heap, saving the stack trace where each object was allocated. Using BCI, this agent incorporates additional byte codes around the object allocations to capture the stack trace and tag the objects that were allocated with that stack trace. As the VM executes your byte code, it also executes the byte code that was added, calling the `Tracker` methods, which will then call the native methods that are registered for the `Tracker` class. Native methods create a `TraceInfo` struct and tag the object with that struct address.

Objects that have non-zero tags are treated differently. Tags are necessary for any objects you are concerned with -- in this case, all objects. Only objects with tags will be seen in any `JVMTI_EVENT_OBJECT_FREE` event, so this is the only way that an object is currently allocated. To have a unique identification per object would require a unique tag value for every object. You could, for example, tag an object with an integer counter, but you would have only that counter, which represented when in allocation time an object was allocated. But you could capture more specific data if that counter were used to index additional data about an object. You could use the counter technique to track details about every 10th object, for example, or perhaps the last 1000 objects allocated. There are many possibilities.

A fairly common and low-impact shortcut when using JDK 6 is to tag only the actual `Class` objects and then use a JVM TI call to `FollowReferences` (new in JDK 6) to quickly sum up the object counts based solely on the type of classes they are. In this case, BCI is not needed, but you won't know where the actual objects were allocated, just that they were allocated. Tagging only the `Class` objects creates very little overhead.

The VM's garbage collector manages allocations by compacting, rearranging, and doing whatever is necessary to reclaim space and provide the space needed for allocations. In the process, objects get moved, which is why having a particular address in the process memory is not very helpful and why tags are used. If you want access to a tagged object, you can get the JNI object handle to an object with `GetObjectsWithTags`, and you can use any of the JNI or JVM TI calls to access that object through that JNI handle.

So how do you tag an object? There are multiple ways. You can use the explicit `SetTag` interface, and you can also simply assign the tag during some of the callbacks from interfaces such as `IterateOverHeap`. Both of these tagging mechanisms are used in the `heapTracker.c` example.

## BCI and BCI Events

This example agent uses a native BCI library called `java_crw_demo` (`libjava_crw_demo.so` or `java_crw_demo.dll`) that is available as part of the JDK downloads in the `demo/jvmti/java_crw_demo` directory.

The `java_crw_demo` function provides for very simple byte-code instrumentation (BCI). It's acceptable for some very basic needs such as these JVM TI demo agents and tools such as HPROF, but it has its limitations. The `java_crw_demo` function provides basic instrumentation of method entries, method returns, new byte codes, and new array byte codes. It's written in C and has been used in several of the JVM TI demo agents and in HPROF. The byte codes injected are limited to simple dups and `invokestatic` byte codes to the methods of a `Tracker` class.

Let's look once more at the `HeapTracker` class:

```
public class HeapTracker {

    // The static field that controls tracking
    private static int engaged = 0;

    // Calls to this method will result in a call into the agent.
    private static native void _newobj(Object thread, Object o);

    // Calls to this method are injected into the class files.
    public static void newobj(Object o) {
        if ( engaged != 0 ) {
            _newobj(Thread.currentThread(), o);
        }
    }

    // Calls to this method will result in a call into the agent.
    private static native void _newarr(Object thread, Object a);

    // Calls to this method are injected into the class files.
    public static void newarr(Object a) {
        if ( engaged != 0 ) {
            _newarr(Thread.currentThread(), a);
        }
    }
}
```

As you can see, there isn't much to this class. The methods `newobj()` and `newarr()` will be called from the injected byte code of the application, which in turn will call the native methods, which have been registered as the functions `HEAP_TRACKER_native_newobj()` and `HEAP_TRACKER_native_newarr()` inside the native agent library. Of course, having the `newobj()` and `newarr()` methods call native methods was an implementation choice, and the developer used it in this case as the quickest way to get back into the native agent library.

The `newobj()` method needs to be called only in the `<init>` method of `java.lang.Object`. Though it is necessary to adjust the `stacktrace` to compensate for the few additional frames, it is an accurate accounting of all objects that are allocated and initialized. The byte-code injection is just a dup and an `invokestatic` byte-code insertion, along with the necessary constant pool entries for the `Tracker` classname and `newobj()` method name. The VM specification does not allow an object to be passed anywhere before it is initialized, so injecting byte codes after every new byte code will trigger verification errors when the object is passed into `newobj()`. An alternative is to find the new byte codes, add the dup after each, and then insert the `invokestatic` byte code after the matching method call for the specific class. Some profilers that use BCI may do this, but this small demo does not.

For more details about how to modify class files, see the source code to the `java_crw_demo` library by downloading either [JDK 5.0](#) or [JDK 6](#).

You can use several BCI class libraries. The JDK provides a way for developers to write pure Java technology-based agents using the `java.lang.instrument` classes and the `-javaagent` option in JDK 5.0. So you are not limited to writing native C or C++ code when doing BCI.

## Conclusion: Countless Solutions to Countless Problems

This article has focused on a particular application of JVM TI for a particular purpose. However, just as there are countless solutions to problems, there are countless varieties of agents that you can write. In general, you will need to experiment and take time to produce a good solution and a good robust agent. To gain better insight into your agent performance, use native tools such as those available in the Solaris 10 OS, DTrace, or [Sun Studio Performance Analyzer](#), a tool that helps assess code performance, identify potential performance problems, and locates where problems occur inside the code.

\* The terms "Java Virtual Machine" and "JVM" mean a Virtual Machine for the Java(TM) platform.

## For More Information

JVM Tool Interface (JVM TI) for [JDK 5.0](#) and [JDK 6](#)  
[Transitioning from JVMPI to JVM TI](#)  
Java Native Interface (JNI) for [JDK 5.0](#) and [JDK 6](#)  
[Kelly O'Hair's Blog](#)  
[Solaris 10 OS DTrace VM Agents Project](#)  
[Sun Studio Performance Analyzer](#)  
[Java Platform Debugger Architecture: Overview](#)  
[Java SE Downloads](#)

## About the Authors

**Kelly O'Hair**, a senior staff engineer at Sun Microsystems, works in the JDK Core Serviceability area and focuses on improving the JDK builds and JVM Tool Interface (JVM TI). Read his [blog](#).

**Janice J. Heiss**, in addition to exploring the world of Java technology as a staff writer for Sun Microsystems, is a published writer of poetry, fiction, and memoir. She has also written and performed for the stage, including stand-up comedy.

## Rate and Review

Tell us what you think of the content of this page.

☐ Excellent ☐ Good ☐ Fair ☐ Poor

Comments:



Your email address (no reply is possible without an address):

[Sun Privacy Policy](#)

Note: We are not able to respond to all submitted comments.

copyright © Sun Microsystems, Inc